UFCD 10788

Fundamentos da linguagem SQL

Objetivos

- Identificar bases de dados relacionais.
- Consultar dados armazenados dentro da base de dados.
- Criar tabelas, inserir, alterar e apagar dados.

Conteúdos

- Introdução a bases de dados
- Ambientes de bases de dados
- Terminologia de bases de dados relacionais
- Planeamento e desenho de bases de dados
- Introdução ao SQL
- Criação de bases de dados
- Tabelas e integridade de dados
- Fundamentos de transact SQL
- Filtragem e ordenação de dados

Base de Dados

- Introdução;
- Conceito de Base de dados;
- Objetivos das Base de dados.
- Vantagens e Desvantagens das Base de Dados.

Introdução

- A partir do momento em que se constatou a enorme quantidade de informação passível de ser tratada pelo computador, e bem assim todo o conjunto de relações que necessariamente se teriam de estabelecer entre as diversas componentes dessa mesma informação, apareceu a noção de Base de Dados.

Conceito de Base de Dados

- Uma Base de dados será, então, um conjunto de dados agrupados como uma só entidade e estruturados de maneira a responder às necessidades de qualquer empresa.

Objetivos das Base de dados

- Os dados estão relacionados entre si, de maneira a simplificar o acesso;
- Os dados podem ser acedidos por utilizadores individualmente, independentemente do conjunto.
- A Base de Dados permite que a empresa tenha um controlo centralizado de informação, que é um dos seus bens mais preciosos.

Vantagens das Bases de Dados

- A redundância pode ser reduzida;
- A inconsistência pode ser evitada;
- Os dados podem ser partilhados;
- A integridade pode ser mantida.

Desvantagens das Bases de Dados

- As desvantagens das Bases de Dados residem essencialmente no seu custo e no tempo que um eficiente sistema de trabalho poderá levar a montar.

Base de Dados Relacional

- uma base de dados relacional é um tipo de base de dados que organiza os dados em tabelas com linhas e colunas, onde cada linha representa uma entidade e cada coluna representa um atributo dessa entidade.
- Essas tabelas são conectadas por meio de relacionamentos definidos entre elas, permitindo a consulta e manipulação eficiente dos dados.
- O modelo relacional, em que as bases de dados relacionais são baseadas, foi introduzido por Edgar F. Codd, e o SQL (Structured Query Language) é a linguagem padrão para interagir com elas.

Sistema de Gestão de Base de Dados (SGBD)

- Definição;
- Conceitos básicos;
- Requisitos fundamentais de um SGBD;
- SGBD VS SGF;
- Utilizadores do SGBD;
- Linguagens de Base de Dados.

Definição de SGBD

- Os programas de bases de dados – frequentemente também designados Sistemas de Gestão de Bases de Dados (SGBD) – permitem, usualmente, criar e trabalhar com múltiplas tabelas e/ou outras unidades de dados ou informação, estabelecer relacionamentos entre essas entidades e efetuar muitas operações de organização da informação que são específicas deste tipo de programas.
- Microsoft Access;
- Oracle;
- Informix;
- Microsoft SQL Server;
- MySQL;
- MariaDB;
- Dbase.

Conceitos básicos

- Redundância;
- Integridade;
- Inconsistência;
- Metadados.

- Redundância:Repetição/replicação desnecessária de dados.
- Integridade:Qualidade dos dados da Base de Dados.
- Inconsistência:Quando as transações não violem nenhuma integridade.
- Metadados:Os metadados são marcos ou pontos de referência que permitem circunscrever a informação sob todas as formas; pode se dizer que são resumos de informações sobre a forma ou conteúdo de uma fonte.

Sistema de Gestão de Base de Dados VS Sistema Gestão Ficheiros

- Abstração dos dados;
- Independência de programadores;
- Partilha de dados;
- Diminuição da redundância;
- Controlo no acesso aos dados;
- Desenvolvimento e manutenção;
- Integridade dos dados;
- Mecanismos de recuperação a falhas;
- Solução mais dispendiosa.

Utilizadores do SGBD

- Administrador da Base de Dados;
- Utilizador final;
- Programador.

Linguagens de Base de Dados

- Linguagem de Definição de Dados (LDD);
- Linguagem de Manipulação de Dados (LMD);
- Linguagem de Controlo de Dados (LCD).

Conceitos

- Tabelas:
- Os dados são armazenados em tabelas, que são como folhas eletrónicas com linhas (registos) e colunas (atributos).
- Relações:
- As tabelas são interconectadas por meio de relacionamentos, que definem como os dados em diferentes tabelas estão ligados. Esses relacionamentos geralmente são estabelecidos por meio de chaves primárias e estrangeiras.
- Chaves:
- As chaves primárias identificam de forma única cada registo em uma tabela. As chaves estrangeiras estabelecem o relacionamento entre tabelas, referenciando a chave primária de outra tabela.
- SQL:
- A Structured Query Language (SQL) é a linguagem padrão para interagir com bancos de dados relacionais, permitindo criar, modificar, consultar e manipular dados

Vantagens das Bases de dados Relacionais

- Estrutura e Organização:
- A estrutura em tabelas facilita a organização e a compreensão dos dados.
- Integridade dos Dados:
- Mecanismos como restrições e validações garantem a consistência e a qualidade dos dados armazenados.
- Flexibilidade:
- A capacidade de definir relacionamentos entre tabelas permite que os dados sejam consultados de várias maneiras, adaptando-se a diferentes necessidades.
- Escalabilidade:
- Bancos de dados relacionais podem ser dimensionados para lidar com grandes volumes de dados e tráfego.
- Transações:
- Suportam transações, garantindo que as operações sejam realizadas de forma atômica, consistente, isolada e durável (ACID), o que é crucial para a integridade dos dados em aplicações críticas.

Exemplos de BD relacionais

- MySQL:
- Um sistema de código aberto popular, amplamente utilizado em diversas aplicações.
- PostgreSQL:
- Outro sistema de código aberto, conhecido por sua robustez e recursos avançados.
- SQL Server:
- Um sistema desenvolvido pela Microsoft, frequentemente usado em ambientes corporativos.
- Oracle:
- Um sistema comercial amplamente utilizado em grandes empresas, com recursos avançados de segurança e desempenho.
- Azure SQL Database:
- Um serviço de banco de dados relacional na nuvem oferecido pela Microsoft.

Aplicações

- Sistemas de Gestão de Conteúdo (CMS): Armazenamento e organização de conteúdo para websites e aplicações.
- Sistemas de Gestão de Relacionamento com o Cliente (CRM): Gerenciamento de informações de clientes e interações.
- Sistemas de E-commerce: Gerenciamento de produtos, pedidos e informações de clientes.
- Sistemas Financeiros: Gerenciamento de transações, contas e informações financeiras.
- Sistemas de Logística: Gerenciamento de inventário, remessas e entregas.

Em resumo

- As bases de dados relacionais são uma ferramenta fundamental para a gestão de dados estruturados, oferecendo uma combinação de organização, flexibilidade, integridade e escalabilidade que as tornam adequadas para uma ampla gama de aplicações.

Chave Primária

- é um campo ou conjunto de campos que identifica de forma única cada registo em uma tabela.
- É fundamental para garantir a integridade e a organização dos dados, permitindo a referência a registros específicos.

Características

- Unicidade: Cada valor da chave primária deve ser único em toda a tabela.
- Não nula: A chave primária não pode conter valores nulos (ausentes).
- Simplicidade: Geralmente, é preferível usar uma única coluna como chave primária, mas pode ser uma combinação de colunas (chave composta).
- Estabilidade: A chave primária não deve mudar com frequência, pois pode afetar relacionamentos com outras tabelas

Funções

- **Identificação única:**
- Permite identificar cada registro de forma exclusiva, como um número de identificação ou CPF.
- **Relacionamentos:**
- É utilizada para estabelecer relações com outras tabelas através da chave estrangeira.
- **Integridade dos dados:**
- Garante que não existam registros duplicados e que os dados sejam consistentes.

Exemplos

- Exemplo:
- Em uma tabela de clientes, o número de identificação do cliente ou o CPF poderiam ser usados como chave primária, pois são valores únicos para cada cliente.
- Em resumo, a chave primária é um conceito essencial em bancos de dados, pois garante a organização, a identificação e a integridade dos dados em uma tabela.

TABELA CLIENTES

ID_CLIENTE	NOME
1	DANIEL
2	JOAO
3	PAULO

PK

TABELA VENDAS

IDCLIENTE	PRODUTO	QUANTIDADE
1	AGUA	1
1	BISCOITO	2
1	CHOCOLATE	1
2	AGUA	2

FK

Chave Externa ou estrangeira

- é um campo em uma tabela que faz referência à chave primária de outra tabela.
- Ela estabelece um relacionamento entre as duas tabelas, garantindo a integridade referencial dos dados.
- A chave estrangeira garante que os valores em um campo de uma tabela correspondam aos valores existentes na chave primária da tabela referenciada.
- Exemplo:
- Imagine duas tabelas: "Clientes" e "Pedidos".
- A tabela "Clientes" tem uma chave primária chamada "ID_Cliente". A tabela "Pedidos" tem uma chave estrangeira chamada "ID_Cliente" que faz referência à chave primária da tabela "Clientes".
- Isso significa que cada pedido deve estar associado a um cliente existente na tabela "Clientes".

Funções

- **Estabelecer relacionamentos:**
- Permite criar conexões entre diferentes tabelas, facilitando a consulta e a análise de dados relacionados.
- **Garantir a integridade referencial:**
- Impede que você adicione dados em uma tabela que não tenham uma correspondência válida na tabela relacionada. Por exemplo, não é possível adicionar um pedido com um ID de cliente que não exista na tabela de clientes.
- **Manter a consistência dos dados:**
- Ao garantir que os relacionamentos sejam válidos, a chave estrangeira ajuda a manter a integridade e a consistência dos dados em todo o banco de dados.
- Em resumo: A chave estrangeira é um conceito fundamental em bancos de dados relacionais, que permite criar e manter relacionamentos entre tabelas, garantindo a qualidade e a consistência dos dados.

Exercícios

- Escola de formação profissional
- Registo de clientes num site

- Elaborar as tabelas, definir os campos, chave primária e chave externa

- Registar as tabelas em folha de cálculo excel
- Filtrar dados

Utilização do MySQL

- O MySQL é utilizado em milhões de páginas web em todo o mundo.
- É utilizado em empresas e organizações como a NASA, Adobe, Linkedin, Tumblr, Casa Branca (EUA), Organização das Nações Unidas, para a gestão das suas bases de dados.

Características do MySQL

- Ficou bastante popular devido à sua fiabilidade, robustez e facilidade de integração com a linguagem PHP;
- É open source;
- É escalável, rápido e seguro;
- Opera em ambiente cliente-servidor;
- Possui um ambiente gráfico agradável e intuitivo;
- Atualmente é um dos maiores e melhores SGBD do mundo.

Instalação do software

- Download mysql
- Visual studio redestribute2019
  - Instalar visual studio
  - Downloads https://www.mysql.com/downloads/
  - Mysql comunnityserver MySQL Community (GPL) Downloads »
    - 9.3.0
    - MySQL Workbench
  - Password: Teste_1!

Versões do MySQL

- Existem versões para vários sistemas operativos (Microsoft Windows, Linux e Mac OS X).
- Existem versões pagas e as gratuitas.
- A versão gratuita é a Community Version, sob a licença GPL.
- O resumo das versões encontra-se em: http://www.mysql.com/products/
- No site http://dev.mysql.com/downloads/ estão as versões sob licença GPL. Estas não são pagas e podem ser utilizadas de forma gratuita. As restantes são pagas.

Conteúdos

- Apresentação da Linguagem SQL;
- Apresentação do Sistema de Gestão de Base de Dados (MySQL);
- Criação, mostrar, utilização e eliminação de Base de Dados em SQL;
- Criação, alteração e eliminação de tabelas e índices em SQL;
- Conceito de transação (comandos COMMIT e ROLLBACK).

História da Linguagem SQL

- A primeira versão da linguagem SQL, chamada SEQUEL (Structured Query English Language - Linguagem de Consulta Estruturada em Inglês), surgiu no início dos anos 70, nos laboratórios da IBM com o objetivo de demonstrar a viabilidade da implementação do modelo relacional.
- Rapidamente outras empresas começaram a trabalhar em paralelo e desenvolveram as suas próprias versões da linguagem.

Continuação História da Linguagem SQL

- A expansão deste conceito levou à necessidade de ser criado um padrão para a linguagem.
- Em 1986, o ANSI (American National Standarts Institute) padronizou a linguagem, sendo hoje conhecida por SQL.

Versões da Linguagem SQL

- SQL-86 (versão standart em 1986)
- SQL-92 (revisão em 1992)
- SQL-99 ou SQL3 (revisão em 1999)
- SQL:2003 (revisão em 2003)

Características da Linguagem SQL

- SQL = Structured Query Language
- Linguagem declarativa usada para definição e manipulação de dados em bases de dados relacionais;
- Com SQL, o programador especifica aquilo que quer, em vez de especificar como se pode obter aquilo que se quer.
- O SGBD é responsável por decidir qual é a melhor forma para obter o que foi pedido.

Tipos de linguagens associadas ao SQL

- DDL – Data definition Language;
- DML – Data Manipulation Language;
- DCL – Data Control Language.

DDL – Data Definition Language

- Todos os comandos DDL são estruturais.
- Comandos que permitem operar sobre a estrutura da Base de dados e das suas tabelas.

- DML: Comandos que permitem operar sobre a informação existente nas tabelas.
- DCL: Comandos que permitem a definição da política de privilégios sobre a informação existente numa base de dados.

DDL – Data Definition Language

- Todos os comandos DDL são estruturais.
- Permitem operar sobre a estrutura da Base de dados e das suas tabelas.

Comandos DDL – Base de Dados

- **Create Database** – Criar Base de Dados;
- **Show Databases** – Mostra Base de Dados;
- **Use** – Usar a Base de Dados;
- **Drop Database** – Eliminar Base de Dados.

Criar Base de Dados

- CREATE DATABASE nome_BD_a_criar;
- Exemplo: CREATE DATABASE agenda;

Listar Base de Dados

- SHOW databases;

Usar a Base de Dados

- USE nome_BD_a_usar;
- Exemplo: USE agenda;

- Eliminar
- DROP DATABASE nome_Base_de_Dados;
- Exemplo: DROP DATABASEagenda;

Comandos DDL - Tabelas

- **Create Table** – Criar tabelas;
- **Show tables** – Listar tabelas;
- **Describe** – Listar os campos da tabela;
- **Alter Table** – Alterar a estrutura de uma tabela;
- **Drop Table** – Eliminar uma tabela.

Tipos de Dados em SQL

- Integer – Números inteiros;
- Float – Números reais;
- Char – Caracteres;
- Varchar – Texto;
- Date – Data;
- Time – Hora.

Criar Tabela

CREATE TABLE nomes(nomes_ID integer(2), nomes _NOME varchar(40), nomes_local varchar(40));

Inserir valores

```
INSERT INTO nomes (nomes_ID, nomes_nome,
nomes_local  VALUES ('10', 'Alexandra Gonçalves', 'Sintra'));
```

- SHOW tables;

Listar campos :DESCRIBE nome_tabela;

- Exemplo: DESCRIBE clube;

- Alterar estrutura da tabela
- O comando ALTER TABLE permite quatro tipos de alterações: ADD, MODIFY, DROP e CHANGE.
- ALTER TABLE nome_tabela tipo_de_alteração;
- Modificar o tipo de dados de um campo:
- ALTER TABLE nome_tabela modify tipo_de_dados;
- Exemplo1: ALTER TABLE jogador modify clube varchar(90);

Realizar consultas à base de dados

- **Select ….from…;**
- Select * from nomes;

Apagar registos de uma tabela

- Delete from …. where...

- Delete from nomes where nomes_id="11";

Editar / Atualizar registos

- UPDATE nome_tabela SET coluna1 = valor1, coluna2 = valor2 WHERE condição;
- UPDATE nomes SET agendaT_nome = "Alexandra Gonçalves" WHERE agendaT_nome="Alexandra"

Primary Key

- CREATE TABLE Pessoas (
  ID int (2)NOT NULL,
  Apelido varchar(255) NOT NULL,
  Nome varchar(255),
  idade int(2),
  PRIMARY KEY (ID)

Chave Externa (foreign key)

```
CREATE TABLE fornecedores (
    FornecedorID int NOT NULL,
    NrEncomenda int NOT NULL,
    IDCliente int,
    PRIMARY KEY (FornecedorID),
    FOREIGN KEY (PersonID) REFERENCES clientes(clientesID
)
);
```

Order By

- ```
  SELECT * FROM disciplinas
  ORDER BY nome ASC;
  ```
- ```
  SELECT * FROM disciplinas
  ORDER BY nome DESC;
  ```

Operadores

=	Equal	igual
>	Greater than	Maior que
<	Less than	Menor que
>=	Greater than or equal	Maior ou igual que
<=	Less than or equal	Menor ou igual que
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=	diferente
BETWEEN	Between a certain range	entre
LIKE	Search for a pattern	Como um
IN	To specify multiple possible values for a column	Na lista de

- **SELECT * FROM nomes WHERE localidade LIKE 'S%';**
- **SELECT * FROM cursos WHERE id-curso <> 2;**
- SELECT * FROM notas WHERE id_nota BETWEEN 12 AND 14;
- SELECT * FROM notas WHERE id_nota <= 12,5;

SQL Agregate Functions

- Min()
- Max()
- Count()
- Avg()
- Sum()

- ```
  SELECT MIN(nota)
  FROM notas;
  ```
- ```
  SELECT MAX(nota)
  FROM notas;
  ```
- ```
  SELECT MAX(nota) AS maior_nota FROM Notas;
  ```
- ```
  SELECT AVG(nota)
  FROM notas;
  ```
- ```
  SELECT SUM(nota)
  FROM notas;
  ```
  ```

INNER JOIN

```
SELECT cursos._idcursos,duraçao, alunos.id_aluno, alunos_nomes,
alunos_data_nascimento
FROM alunos, cursos
INNER JOIN cursos ON cursos.cursos_ID=alunos.alunosID_curso;
```

- **Liste o nome dos alunos e o nome da turma a que pertencem.**
- SELECT A.nome, T.nome_turma
- FROM Alunos A
- JOIN Turmas T ON A.id_turma = T.id_turma;

- **Mostre o nome dos professores que ensinam a turma “9ºA”.**

- SELECT DISTINCT P.nome
- FROM Professores P
- JOIN Turmas_Professores TP ON P.id_professor = TP.id_professor
- JOIN Turmas T ON TP.id_turma = T.id_turma
- Liste os alunos nascidos depois de 2010.
- SELECT * FROM Alunos WHERE data_nascimento > '2010-01-01';